

The Cost of Memory Access: Direct, Vector, and Indirect Summation on Perlmutter CPU Nodes

Coding Project 1, CSC 746, Fall 2026

Ryan Selvarajan
Computer Science Department
San Francisco State University
San Francisco, CA, USA
Email: rselvarajan@sfsu.edu

Abstract—We implement three summations performing identical work with N 64-bit additions: a direct sum touching no array, a vector sum reading sequentially, and an indirect sum reading through random indices. The goal is to show the difference in the memory access patterns and the computing performance. Each is instrumented with `std::chrono` and evaluated at six problem sizes from 64 MB to 2 GB on a single core of a Perlmutter CPU node (AMD EPYC 7763). The metrics involved include MFLOP/s, memory bandwidth utilization, and average latency. Sequential access proves to be nearly free, reaching 88% of the arithmetic-only rate; random access costs 16× and degrades further as the problem grows.

I. INTRODUCTION

A processor can do arithmetic far faster than memory can supply the numbers to work on. Because of that, how a program reaches its data often matters more than how much arithmetic it does, and counting operations turns out to be a poor way to predict how long code will run. That cost is easy to state in general terms but hard to guess at for any particular program, so it is worth measuring directly.

We study this with a benchmark harness built around one driver program and three interchangeable implementations. The driver allocates the data, steps through a range of problem sizes, and times each summation, while each implementation supplies its own version of the two routines the driver calls. Because the driver is shared and only the implementation changes, the three programs differ in nothing but the loop that does the summing.

Each program is run at a range of problem sizes, all large enough that the data cannot fit in cache, and repeated several times so that one slow run cannot skew the result. The version that never touches memory sets the upper limit on speed, the version that reads memory

in order comes close to matching it, and the version that reads at random falls well behind and slips further as the data grows. The insight is that arithmetic alone cannot explain this: all three perform the same additions, so the entire spread comes from where the data sits and the order in which it is read.

II. IMPLEMENTATION

This section describes the three summation programs. All three are built against a single benchmark program allocates the working array, steps through the problem sizes, and times each summation, so the resulting executables differ only in the routine under test. Section A presents the direct sum, which performs the additions without touching memory and establishes an arithmetic only baseline. Section 2 presents the vector sum, which reads the same number of values from a contiguous array in index order. Section 3 presents the indirect sum, which reads through randomly generated indices and so gives up any predictable access pattern.

A. Direct Sum

The objective of direct sum is to establish how fast the processor performs the additions when memory is unused. It computes the same number of additions as the other two implementations but generates the values it adds rather than reading them, so its runtime reflects arithmetic throughput alone.

Because there is no data to prepare, `setup()` is empty; it accepts the parameters required by the shared interface and returns immediately. The `sum()` routine, shown in Listing 1, initializes a 64-bit accumulator to zero and adds the loop counter to it on each of N iterations, so the values being summed are produced by the loop itself rather than fetched from anywhere.

```

1 // Direct Sum
2 void
3 setup(int64_t N, int64_t A[])
4 // N: number of values. A: the array (unused)
5 {
6     // no setup required, so function is empty
7 }
8 int64_t
9 sum(int64_t N, int64_t A[])
10 // N: # of values to add.
11 // A: the array (unused here)
12 {
13     int64_t dsum = 0; // 64-bit integer
14     for (int64_t i = 0; i < N; i++) {
15         // loops for N-1 iterations
16         dsum += i; // add i to the running total
17     }
18     return dsum; // returns total to caller
19 }

```

Listing 1. Direct sum: accumulates the integers 0 through N-1 without touching the array, giving an arithmetic-only baseline.

B. Vector Sum

The objective of the vector sum is to measure the cost of reading the values from memory rather than generating them. It performs the same N additions as the direct sum, but each addend is fetched from an array that is laid out contiguously and read from beginning to end.

The setup() routine fills the first N elements of the array with the values 0 through N-1, writing them in index order. The sum() routine, shown in Listing 2, then walks the same array from index 0 to N-1, adding each element to a running total. The only difference from the direct sum is where the addenda comes from: sum += A[i] in place of sum += i.

C. Indirect Sum

The objective of the indirect sum is to measure what happens when the order of memory accesses becomes unpredictable. It performs the same N additions as the other two implementations, but the location of each value is decided by data read at run time rather than by the loop counter. Because it reads from the same array as the vector sum and differs only in the order it visits, the gap between the two isolates the cost of losing locality.

The setup() routine fills the array with random values in the range 0 to N-1, so every entry is simultaneously a stored value and a valid index into the array itself. The sum() routine, shown in Listing 3, reads A[i] to obtain an index and immediately uses that index to read a second element, accumulating A[A[i]]. Each iteration therefore performs two reads instead of one: the first

```

20 // Vector Sum
21 void
22 setup(int64_t N, int64_t A[])
23 // N: number of values A: the array to fill
24 {
25     for (int64_t i = 0; i < N; i++)
26         // loops for N-1 iterations
27         A[i] = i;
28     // store i into slot i of the array
29 }
30 int64_t
31 sum(int64_t N, int64_t A[])
32 // N: how many values to add.
33 // A: the array to read
34 {
35     int64_t vsum = 0; // running total
36     for (int64_t i = 0; i < N; i++)
37         // loops for N-1 iterations
38         vsum += A[i];
39     // adds the value stored in slot i
40     return vsum; // sends total back to caller
41 }
42 }

```

Listing 2. The vector sum. setup() fills a contiguous array with the values 0 through N-1, and sum() reads them back in index order, so every addend comes from memory rather than from a register.

sequential and predictable, the second landing at an arbitrary position in a buffer that reaches 2 GB at the largest problem size.

III. EVALUATION

This section presents three experiments, each applying one performance metric to all three implementations across the six problem sizes given in Sec. III-B. The first measures sum rate in MFLOP/s, establishing how fast each implementation completes identical arithmetic; the second measures memory bandwidth utilization, showing what fraction of the machine’s data-movement capacity each one consumes; the third measures average memory latency, giving the amortized cost of a single access. Taken together, the three separate the cost of performing the arithmetic from the cost of reaching the data, and show how that separation widens as the problem size grows.

A. Computational platform and Software Environment

All measurements were taken on a single CPU node of NERSC’s Perlmutter system, which contains two AMD EPYC 7763 processors with 64 cores each, running at a 2.45 GHz base clock and a 3.5 GHz maximum boost. Each core has a private 32 KB L1 data cache and a 512 KB L2 cache, while the 256 MB of L3 on a socket is partitioned across eight core complexes,

```

43 // Indirect Sum
44 void
45 setup(int64_t N, int64_t A[])
46 // N: how many values. A: the array to fill
47 {
48     for (int64_t i = 0; i < N; i++)
49         // loops for N-1 iterations
50         A[i] = lrand48() % N;
51         // random value in the range 0 to N-1
52         // % = remainder
53 }

55 int64_t
56 sum(int64_t N, int64_t A[])
57 // N: how many values to add. A: the array to
58 // read
59 {
60     int64_t isum = 0; // running total
61     for (int64_t i = 0; i < N; i++)
62         // loops for N-1 iterations
63         isum += A[A[i]];
64         // read A[i] to get an index,
65         // then add the value stored there
66     return isum; // sends total back to caller

```

Listing 3. The indirect sum. `setup()` fills the array with random values in the range 0 to $N-1$, and `sum()` uses each of those values as an index back into the same array, so every iteration performs one sequential read followed by one unpredictable one.

so a single-threaded program reaches only the 32 MB slice belonging to its own complex. The node provides 512 GB of DDR4-3200 memory, giving a theoretical peak bandwidth of 204.8 GB/s per socket, and runs SUSE Linux Enterprise Server 15; all three programs are serial and were run on one core of a dedicated compute node obtained through an interactive allocation. The codes were compiled with GNU g++ 7.5.0 under CMake’s Release configuration using `-std=gnu++11 -O1 -Wall -pedantic -march=native`, with the optimization level fixed at `-O1` by the supplied `CMakeLists.txt`, which keep the accumulator in a register without the aggressive loop restructuring that `-O3` would apply unevenly across the three implementations.

B. Methodology

The primary measurement is elapsed wall-clock time. Instrumentation added to `benchmark.cpp` records a timestamp and immediately before the call to `sum()` and a second timestamp immediately after. The difference between the two is stored as a `std::chrono::duration<double>` and reported in seconds. Only the summation is timed. `setup()` executes outside the timed region, so the cost of initializing the array

never appears in any measurement. Three metrics are derived from the elapsed time t and the problem size N :

$\text{MFLOP/s} = (N / 10^{**6}) / t$, $\text{bandwidth} = [(\text{bytes} / t) / 204.8\text{e9}] \times 100$, $\text{avg latency} = (t / \text{accesses}) \times 10^{**9}$ [nanoseconds]

Every implementation performs one addition per element, so $\text{ops} = N$ in all three cases and the MFLOP/s formula is identical for each. The byte and access counts differ, because the three codes touch memory in different amounts.

The vector sum reads one 8-byte integer per element. The indirect sum reads two: $A[i]$ to obtain an index, then $A[A[i]]$ to obtain the value. The direct sum reads none. Because the direct sum performs no memory accesses, its bandwidth utilization is 0 percent and its average latency is undefined; the latter is reported as N/A rather than zero, since zero would imply infinitely fast memory rather than the absence of memory traffic.

Two conversions are applied that the metric definitions leave implicit. The bandwidth expression yields a fraction, so it is multiplied by 100 to give a percentage. The latency expression yields seconds, so it is multiplied by 10^{**9} and reported in nanoseconds. The capacity term is 204.8 GB/s, the peak bandwidth of one EPYC 7763 socket, chosen because the code is single-threaded and runs on one core of one socket.

Tests were run over the six problem sizes hard-coded in `benchmark.cpp`. The largest is 2^{**28} elements and the remaining five are that value successively halved down to 2^{**23} , so each size is double the one before it. Each element is a 64-bit integer, giving a working array that spans 64 MB at the smallest size and 2 GB at the largest. Every size exceeds the 32 MB of L3 cache available to a single core, so all measurements fall in the regime where main memory rather than cache supplies the data.

The three executables were built once and run on a dedicated Perlmutter CPU node obtained through an interactive allocation. Each was run three times, with a single run sweeping all six problem sizes in increasing order and calling `setup()` to refill the array before each timed summation. The median of the three runs is reported for each implementation and problem size, the median was chosen over the mean because one indirect-sum run at the largest size was 11 percent slower than the other two and would have skewed an average. Correctness was verified before any timing was recorded: the direct and vector sums both matched the closed form $N(N-1)/2$ exactly at all six sizes, and the indirect sum, which has no closed form, was confirmed by all three runs producing byte-identical totals.

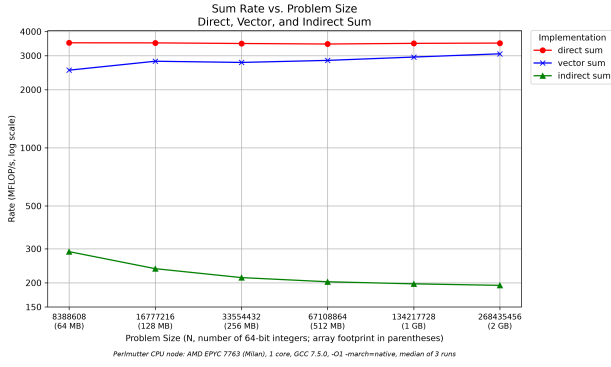


Fig. 1. Sum rate in MFLOP/s versus problem size on a logarithmic vertical axis, showing the direct sum flat near 3500 MFLOP/s, the vector sum reaching 88% of that ceiling at the largest size, and the indirect sum an order of magnitude lower and declining as N grows.

C. MFLOPs

This experiment asks how quickly each implementation completes its additions, and whether that rate depends on problem size. Because all three perform exactly N additions, MFLOP/s compares them on identical work, so any separation between the curves is attributable to memory behavior rather than to the amount of arithmetic performed. Each implementation was timed at the six problem sizes described in Sec. III-B, from 8,388,608 to 268,435,456 elements, with the rate computed as $(N/10^6)/t$ from the median of three runs. Figure 1 plots the result on a logarithmic vertical axis, which is necessary because the fastest and slowest rates differ by more than an order of magnitude.

D. Percentage of Memory Bandwidth

This experiment asks what fraction of the machine’s available memory bandwidth each implementation actually consumes. The three codes request different amounts of data none, $8N$ bytes, and $16N$ bytes as stated in Sec. III-B. So the comparison shows whether requesting more bytes translates into moving more bytes per second. Utilization was computed as $[(\text{bytes}/t)/204.8 \times 10^9] \times 100$ against the peak bandwidth of one EPYC 7763 socket, using the same six problem sizes and median timings. Figure 2 plots the result on a linear vertical axis, since the direct sum’s value of zero cannot be represented on a logarithmic one.

E. Memory Latency

This experiment asks what a single memory access costs on average in each implementation, and whether that cost changes as the working set grows. The metric divides elapsed time by the total number of accesses,

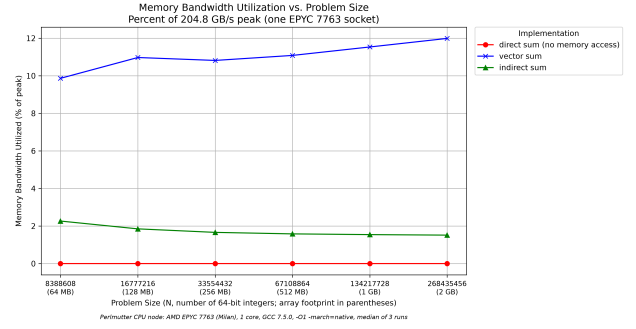


Fig. 2. Memory bandwidth utilization as a percentage of the 204.8 GB/s peak of a single EPYC 7763 socket, showing the vector sum rising from 9.9% to 12.0%, the indirect sum falling from 2.3% to 1.5%, and the direct sum at zero because it performs no memory accesses.

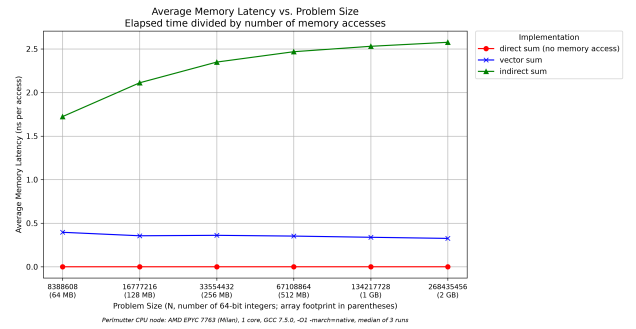


Fig. 3. Average memory latency, computed as elapsed time divided by the number of memory accesses, showing the vector sum steady near 0.35 ns per access, the indirect sum climbing from 1.72 to 2.58 ns, and the direct sum at zero as it performs no accesses.

N for the vector sum and $2N$ for the indirect sum, stated in Sec. III-B. So it reports an amortized per-access cost rather than the latency of any individual load. Because the direct sum performs no accesses the quantity is undefined for it, and it is plotted as zero. The same six problem sizes and median timings are used, and Figure 3 reports the result in nanoseconds per access.

ACKNOWLEDGEMENT

AI tools were used during the preparation of this assignment in these ways. To build understanding of the provided code harness, the C++ constructs it uses, to diagnose build and runtime errors encountered during development, analyze all data and output when running the machine and to check grammar and improve the flow of the written report. All code was compiled, executed, and verified, and all performance data reported here was collected from my own runs on the Perlmutter CPU node.